

Investigating Techniques of Monte Carlo Integration

Charles Giulian

I. Background

Monte Carlo integration is a numerical technique for solving definite integrals using random numbers. While other numerical techniques (like the trapezoidal rule or Riemann sums) can be used to solve integrals deterministically, Monte Carlo integration is unique because it is non-deterministic. Although this means that successive runs of the same Monte Carlo algorithm will produce different results, it makes estimation of higher-order ($d \geq 4$) integrals much faster than quadrature methods. The general principle behind Monte Carlo integration is to “square off” the volume of interest, and then sample many points within that volume to approximate the integral. Differences in algorithms arise from how those random points are sampled. By changing our sampling technique, we can reduce the variance of the Monte Carlo integral and therefore increase accuracy and efficiency.

Given some multidimensional definite integral:

$$I = \int_S f(\vec{x}) d\vec{x}$$

where S , a subset of $\mathbb{R}^m$ has volume:

$$V = \int_S d\vec{x}$$

I has the approximation A where N is the number of samples:

$$I \approx A = V \frac{1}{N} \sum_{i=1}^N f(\vec{x}_i)$$

And by the law of large numbers:

$$\lim_{N \rightarrow \infty} A = I$$

II. Uniform Monte Carlo Integration

Uniform Monte Carlo integration of a one-dimensional function uses a uniform distribution to sample points within V . The following code evaluates

$$\int_{-5}^5 e^{-x^2} dx$$

and then creates a graph to visualize points inside the region of integration (green) and those outside the region of integration (blue). It does this by using `runif(N, min, max)`, which generates N deviates uniformly between `min = a`, `max = b`. Each x and y coordinate is generated independently, and then each coordinate pair is classified as “in” or “out” with respect to the region of integration. We can then approximate the integral by multiplying the ratio of in/out points by the squared-off volume. Comparing our estimation to the analytical solution, which is close to $\sqrt{\pi}$, can inform how accurate this process is. The uniform distribution is the simplest form of Monte Carlo integration, and will serve as a reference point for more efficient approximations in the subsequent sections.

```
# Setup packages
library(ggplot2)
theme_update(plot.title = element_text(hjust = 0.5))
theme_update(plot.subtitle = element_text(hjust = 0.5))
library(rmutil)
```

```
##
## Attaching package: 'rmutil'

## The following object is masked from 'package:stats':
##
##      nobs

## The following objects are masked from 'package:base':
##
##      as.data.frame, units
```

```
invisible(library(cubature))
```

```
## Warning: package 'cubature' was built under R version 4.5.3
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##      filter, lag

## The following objects are masked from 'package:base':
##
##      intersect, setdiff, setequal, union
```

```
library(here)
```

```
## Warning: package 'here' was built under R version 4.5.3
```

```
## here() starts at C:/Users/charl/OneDrive/Documents/montecarlo
```

```
f <- function(x) exp(-x^2)
a <- -5
b <- 5
N <- 50000
x_vals <- seq(a, b, length.out = 1000)
y_max <- f(0)

# Uniform sample between a, b and 0, y_max
df <- data.frame(
```

```

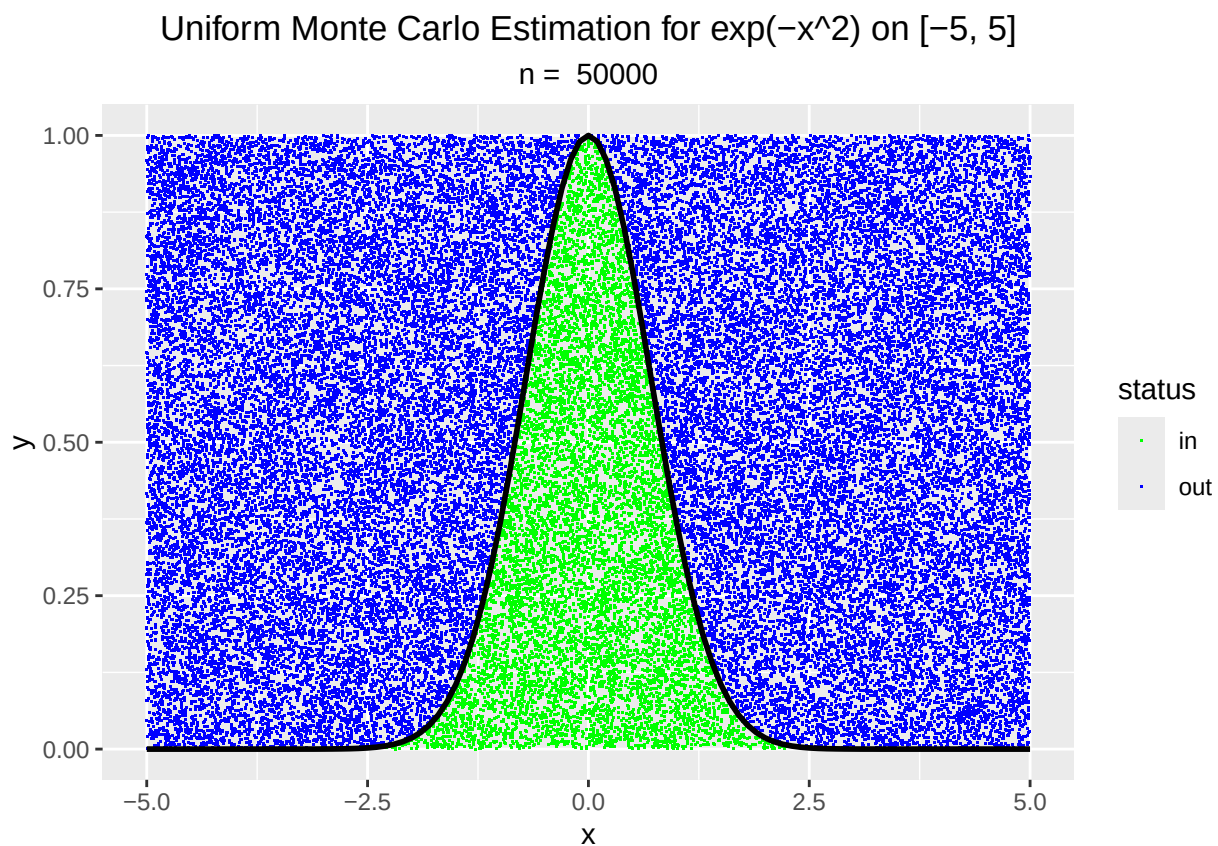
x = runif(N, min = a, max = b),
y = runif(N, min = 0, max = y_max)
)

df$status <- ifelse(df$y < f(df$x), "in", "out")

V <- (b - a) * y_max
I <- (sum(df$status == "in") / N) * V

ggplot(df, aes(x, y)) +
  geom_point(aes(color = status), size = 0.5, alpha = 1, shape = ".") +
  stat_function(fun = f, color = "black", linewidth = 1) +
  scale_color_manual(values = c("in" = "green", "out" = "blue")) +
  labs(title = "Uniform Monte Carlo Estimation for exp(-x^2) on [-5, 5]", subtitle = paste("n = ", N))

```



```
cat(paste("Approximated value of integral:", I, "\n"))
```

```
## Approximated value of integral: 1.7622
```

```
cat(paste("Error from uniform distribution method:", sqrt(pi) - I, "\n"))
```

```
## Error from uniform distribution method: 0.0102538509055159
```

This error value shows that our estimation is pretty close to the analytical solution.

III. Importance Sampling

The next Monte Carlo algorithm to evaluate is importance sampling. While the uniform sampling algorithm above treats every part of the region of integration with the same “weight,” or importance, importance sampling focuses random points where the function is most significant. In the case of our integrand, e^{-x^2} , the function approaches zero as we move further away from the origin. This means that points further away from the origin are less important to the estimation of the integral. Sampling uniformly from $[-5, 5]$ as we did in the previous example plots many points in “empty” regions. Importance sampling uses a “proposal distribution,” called $g(x)$, that mimics the shape of the function we are trying to estimate. This means that to use importance sampling, we need to know something about the shape of the function we are trying to estimate. We estimate the integral using the weight:

$$w_i = \frac{f(x_i)}{g(x_i)}$$

In this section, we use two different proposal distributions: the Normal distribution ($\sigma = 1.5$), and the Laplace distribution ($\mu = 0, b = 2$). These will be effective proposal distributions because, like our integrand e^{-x^2} , they are unimodal and symmetric. Additionally, Normal distribution has smaller tails, while the Laplace distribution has a bigger tail.

```
# 1. Setup
f <- function(x) exp(-x^2)
a <- -5
b <- 5
N <- 50000

# Function to run the simulation and return a plot
get_is_plot <- function(dist_name, x_samples, g_pdf_func) {
  # Calculate weights: f(x) / g(x)
  weights <- f(x_samples) / g_pdf_func(x_samples)
  I <- mean(weights)

  # Generate points
  df <- data.frame(
    x = x_samples,
    y = runif(N, 0, 1)
  )
  df$status <- ifelse(df$y < f(df$x), "in", "out")

  cat(paste(dist_name, "Approximated value of integral:", I, "\n"))
  cat(paste(dist_name, "Error:", sqrt(pi) - I, "\n"))

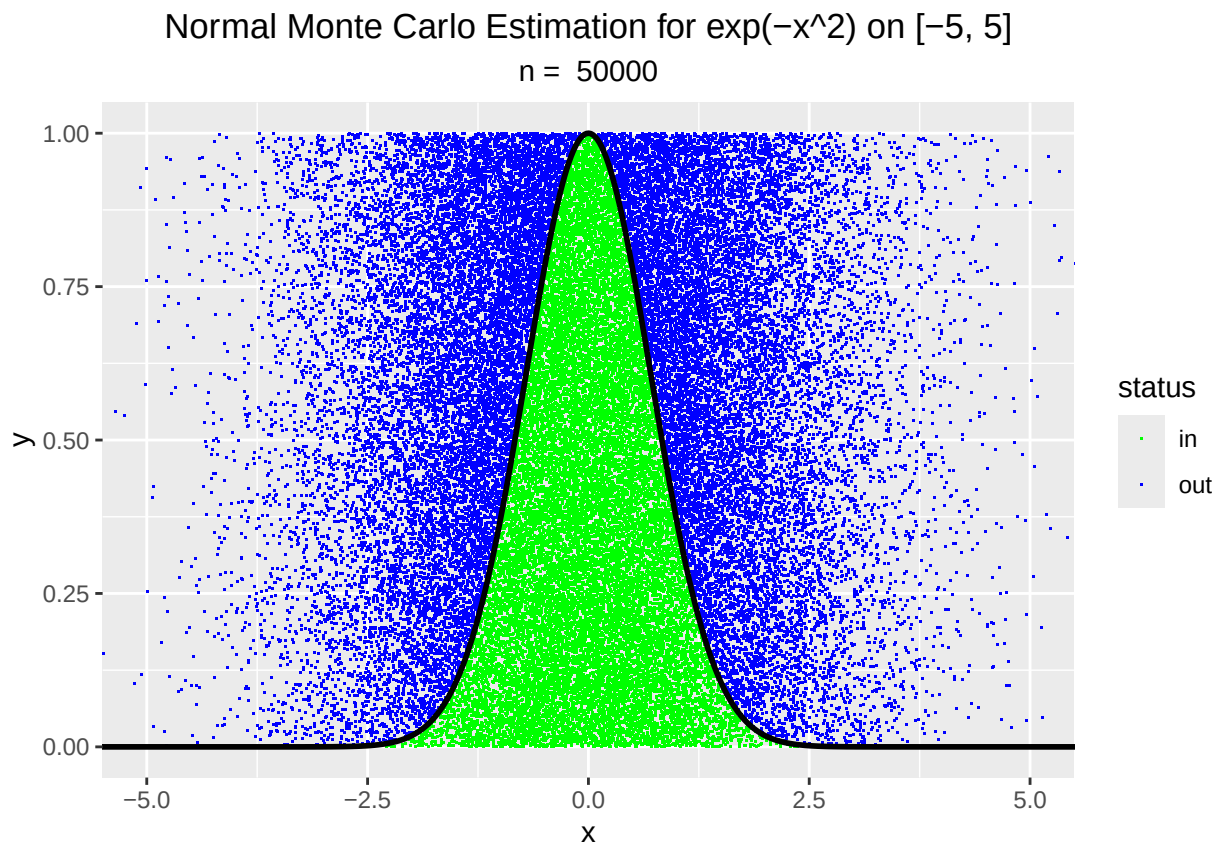
  # Plot results
  print(ggplot(df, aes(x, y)) +
    geom_point(aes(color = status), size = 0.5, alpha = 1, shape = ".") +
    stat_function(fun = f, color = "black", linewidth = 1, n = 500) +
    scale_color_manual(values = c("in" = "green", "out" = "blue")) +
    labs(title = paste(dist_name, "Monte Carlo Estimation for exp(-x^2) on [-5, 5]"), subtitle = paste(
    coord_cartesian(xlim = c(-5, 5)))
  )
}

# Normal
sigma <- 1.5
norm_samples <- rnorm(N, mean = 0, sd = sigma)
```

```
norm_adj <- pnorm(b, 0, sigma) - pnorm(a, 0, sigma)
g_pdf_norm <- function(x) dnorm(x, 0, sigma) / norm_adj

get_is_plot("Normal", norm_samples, g_pdf_norm)
```

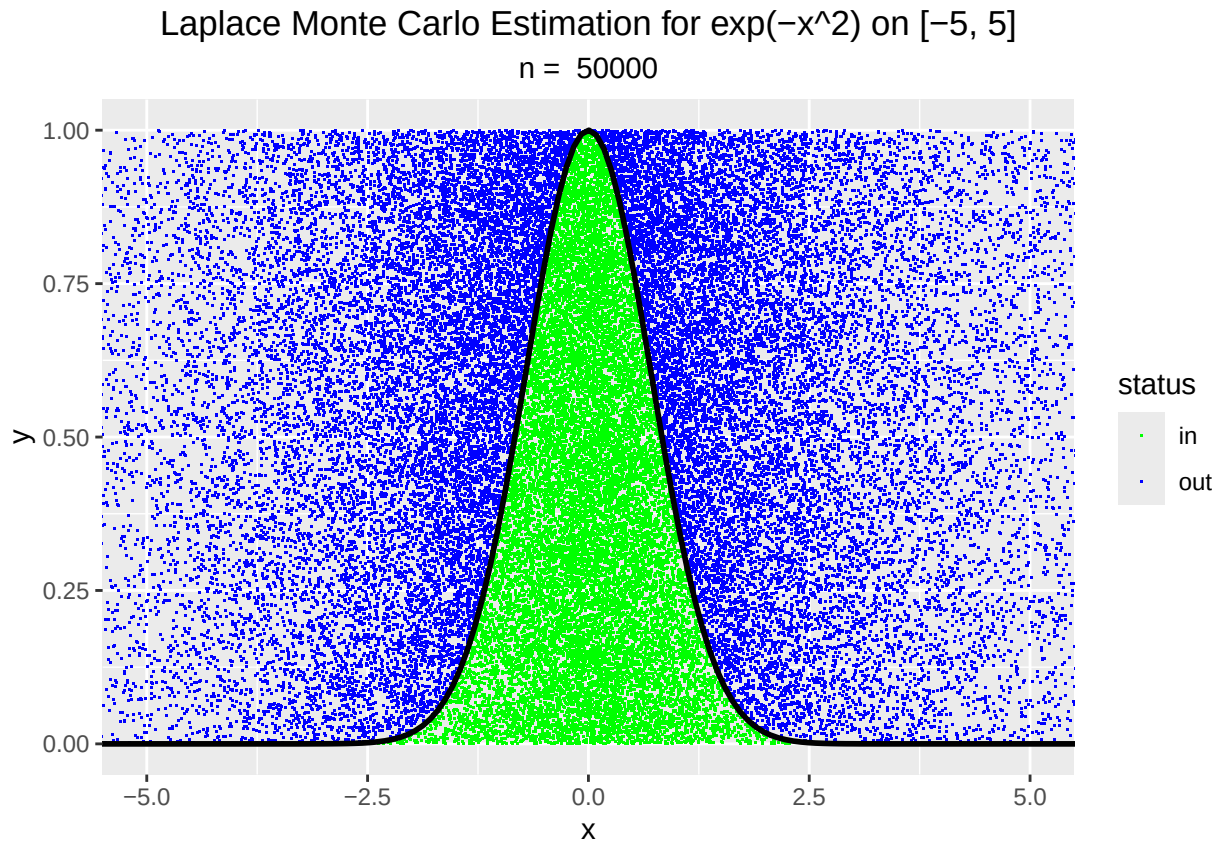
```
## Normal Approximated value of integral: 1.76353466404655
## Normal Error: 0.00891918685896753
```



```
# Laplace
scale_lap <- 2
lap_samples <- rlaplace(N, m = 0, s = scale_lap)
lap_adj <- plaplace(b, 0, scale_lap) - plaplace(a, 0, scale_lap)
g_pdf_lap <- function(x) dlaplace(x, 0, scale_lap) / lap_adj

get_is_plot("Laplace", lap_samples, g_pdf_lap)
```

```
## Laplace Approximated value of integral: 1.61980559998847
## Laplace Error: 0.152648250917043
```



IV. Results

In order to compare the error of the above algorithms, five simulations were run at various N values. The algorithms from previous sections were also compared to a popular VEGAS algorithm provided by the `cubature` package. The VEGAS algorithm, proposed by Lepage in 1978, which is a recursive importance sampling algorithm. This is an advanced technique that combines stratified and importance sampling, results in very low variance.

```
N = 200000
```

```
integrand <- function(arg, weight) {
  x <- arg[1]
  ff <- exp(-x^2)
  return(ff)
}
```

```
I = vegas(integrand, lowerLim = -5, upperLim = 5, maxEval = N)[["integral"]]
cat(paste("VEGAS approximated value of integral:", I, "\n"))
```

```
## VEGAS approximated value of integral: 1.77499820715249
```

```
cat(paste("VEGAS Error:", sqrt(pi) - I, "\n"))
```

```
## VEGAS Error: -0.00254435624696958
```

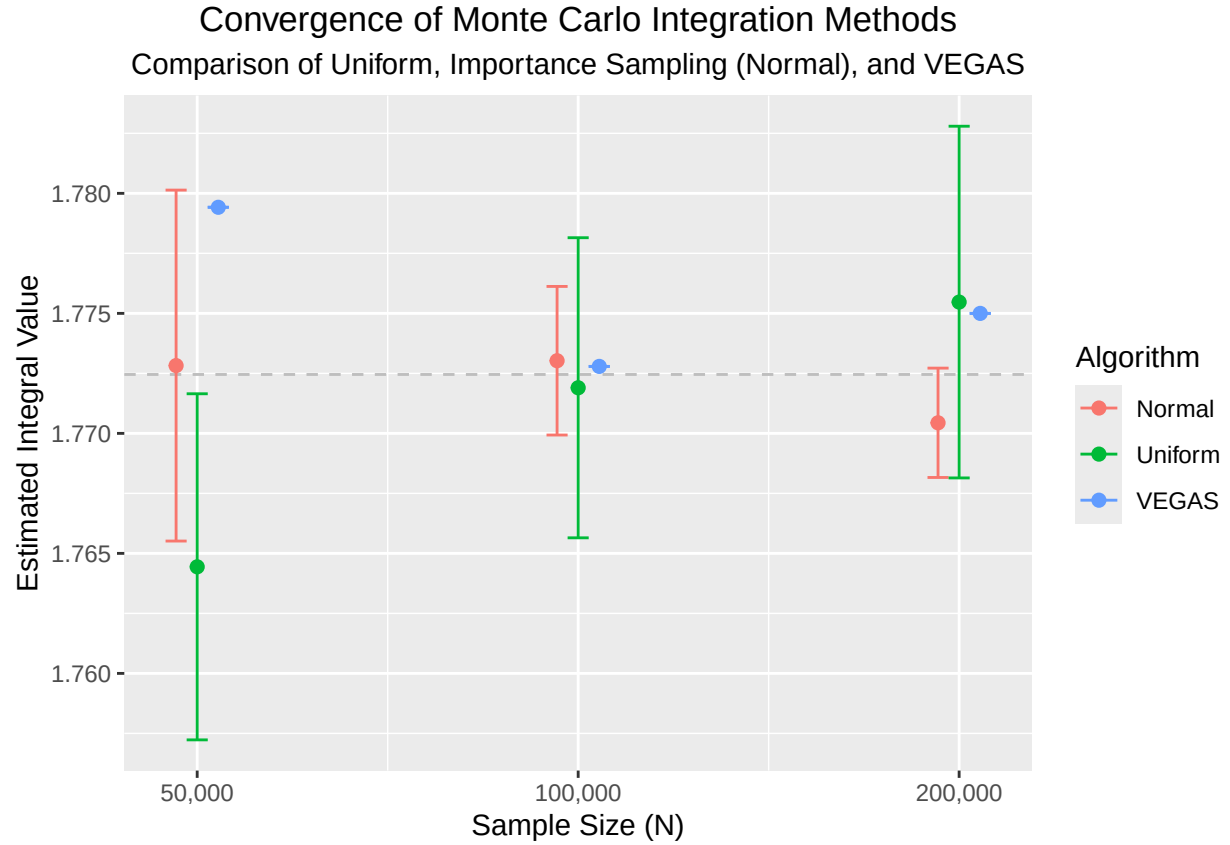
The following graph summarizes the convergence of the Uniform and Normal importance sampling, and VEGAS algorithms at $N = 5e4, 1e5, 2e5$. The Laplace importance sampling algorithm was omitted because it skewed results due to it having the highest error. The graph shows that the VEGAS algorithm had the lowest variability, and that any given algorithm was not significantly more accurate than any other.

```
load(here("results.RData"))

results_summary <- results %>%
  group_by(Algorithm, N) %>%
  summarize(
    Mean_Est = mean(Estimate),
    SD_Est = sd(Estimate),
    .groups = 'drop'
  )

true_val <- sqrt(pi)

ggplot(results_summary, aes(x = N, y = Mean_Est, color = Algorithm, group = Algorithm)) +
  geom_hline(yintercept = true_val, linetype = "dashed", color = "gray") +
  geom_errorbar(aes(ymin = Mean_Est - SD_Est, ymax = Mean_Est + SD_Est), width = 0.05, position = position_dodge(width = 0.05)) +
  geom_point(size = 2, position = position_dodge(width = 0.05)) +
  scale_x_log10(labels = scales::comma) +
  labs(
    title = "Convergence of Monte Carlo Integration Methods",
    subtitle = "Comparison of Uniform, Importance Sampling (Normal), and VEGAS",
    x = "Sample Size (N)",
    y = "Estimated Integral Value",
    color = "Algorithm"
  )
```



V. Acknowledgements and References

Thank you to Dr. Fan Zhang for her help and guidance throughout this project.

Press, William H., et al. Numerical Recipes: The Art of Scientific Computing. 3 ed., Cambridge University Press, 2007.

Lepage, G. Peter. “A New Algorithm for Adaptive Multidimensional Integration.” Journal of Computational Physics, vol. 27, no. 2, 1978, pp. 192-203. ScienceDirect, [https://doi.org/10.1016/0021-9991\(78\)90004-9](https://doi.org/10.1016/0021-9991(78)90004-9).

“Monte Carlo integration.” Wikipedia, https://en.wikipedia.org/wiki/Monte_Carlo_integration.